
TrustLoopGuard for x402 Agentic Payments

Pre-Signing Authorization, Spend Reservation, and Receipts for AI Agents That Can Pay

Duc Minh Nguyen
Founder
duc@gettrustloop.app

July 8, 2026

Abstract

AI agents are crossing the line from conversation into spend. The x402 protocol gives them a native way to pay for APIs, MCP servers, data feeds, and machine-consumable services over HTTP. That makes the payment step easier. It also moves the hard problem one layer up: who proves the agent was allowed to spend, and who is on the hook when it was not?

AP2, Visa Intelligent Commerce, Mastercard Agent Pay, PayPal agentic commerce, and AWS AgentCore Payments are all converging on the same answer: agentic payments need more than a credential and a rail. They need bounded authority, spend caps, payment proof, observability, and dispute evidence.

This paper proposes TrustLoopGuard as the authorization layer for agents that pay. TrustLoopGuard sits between an x402 payment request and wallet signing to authorize a specific agent spend, reserve budget atomically, and produce proof after settlement. It normalizes the payment requirement, binds the spending principal, checks mandate scope, evaluates policy, validates the counterparty, and returns a decision. The wallet signs only if the action is authorized. After settlement, TrustLoopGuard commits the reservation with provider proof or releases it on failure. The result is not just a payment trace. It is the record that explains why this specific agent spend was allowed.

Keywords agentic payments · x402 · AI agents · payment authorization · mandates · spend limits · receipts · runtime security

I. Introduction

Agents can now move from intent to payment. A research agent can pay for a market-data endpoint. A coding agent can pay for a sandboxed tool. A procurement agent can buy a SaaS add-on. A support agent can issue a refund. This is where the system stops being a chat interface and becomes an actor in the business.

That shift creates a new security boundary. If an agent can pay, the important question is not only whether it can reach the endpoint or construct the payment proof. The question is whether there was a stop button before money moved: whether this principal was allowed to spend this amount, through this rail, for this resource, to this counterparty, under this mandate, at this moment.

Thesis. TrustLoopGuard caps and authorizes specific agent payments before the wallet signs, then proves what happened after settlement.

The immediate integration point is **x402**. In the **x402** flow, a client requests a paid HTTP resource, receives a 402 response with payment requirements, signs a payment payload, retries the request with proof, and receives the paid resource after verification and settlement [1, 2, 3]. The protocol makes the paid-resource loop simple enough for agents: request, pay, prove, continue.

But the raw payment loop does not answer enterprise authorization questions. Which service principal is spending? Which user or business mandate funds the action? Is the endpoint allowed? Is the **payTo** address approved? Will parallel payments overspend the same budget? Can the operator prove that the final settlement matched the authorization?

This paper describes TrustLoopGuard as an authorization layer for this action boundary. It does not replace **x402**, wallets, facilitators, or payment processors. It adds a pre-signing decision point and a post-settlement evidence trail:

```
POST /v1/agentive-payments/authorize
POST /v1/agentive-payments/{id}/commit
POST /v1/agentive-payments/{id}/rollback
GET  /v1/agentive-payments/{id}/receipt
```

The interface is intentionally narrow: authorize before signing, commit when settlement proof matches, roll back an unspent reservation when signing or settlement fails, and retrieve the receipt that explains the decision.

II. Market Convergence on Authorization

The agentive payment market is early, but the serious implementations are not converging around "let the agent hold a payment credential." They are converging around the controls next to that credential: identity, authorization, bounded delegation, spend control, proof, and auditability.

AP2 and mandates. Google's Agent Payments Protocol frames agentive payments as an authorization problem. AP2 defines roles such as Shopping Agent, Credential Provider, Merchant, Merchant Payment Processor, and Trusted Surface. It uses checkout mandates and payment mandates to prove that an agent is authorized to purchase and pay for a specific checkout [4]. In human-present flows, the user approves closed mandates. In human-not-present flows, the user approves open mandates with constraints, and the agent later signs closed mandates if the final checkout and payment satisfy those constraints [5]. The

architectural lesson is that an agent should not hold ambient payment authority. It should hold constrained authority that can be verified later.

x402 and machine payments. x402 provides a simple HTTP-native payment loop for paid resources [2, 3]. It is intentionally lightweight: a server returns payment requirements, the client submits payment proof, and a server or facilitator verifies and settles. That is exactly why x402 needs a separate authorization layer in enterprise deployments. The protocol moves value; it does not know every customer’s internal mandate, team budget, approved vendor list, or approval policy.

AWS AgentCore and production controls. AWS AgentCore Payments shows the production control-plane pattern for agent payments. Its technical deep dive describes payment instruments, payment limits, payment orchestration, secure credential handling, x402 support, provider connectors, session budgets, payment proof, and observability [6]. The most important detail is concurrency: agents may initiate several payments in parallel from the same budget, so spending enforcement must reserve budget atomically before provider execution. AgentCore’s CLI documentation also describes an interceptor pattern: catch a 402 response, process payment, attach proof, and retry the request, without exposing payment settlement as a general agent tool [7].

Networks and PSPs. Visa Intelligent Commerce emphasizes trusted agent interactions, credential controls, authentication, fraud monitoring, and merchant visibility [8]. Visa’s Trusted Agent Protocol focuses on merchant-specific, purpose-specific, time-bound signatures that resist replay and help merchants distinguish trusted agents from malicious automation [9]. Mastercard Agent Pay and Agent Pay for Machines emphasize tokenized agent payments, credentialing, permissioning, spending limits, and multi-rail settlement [10, 11]. PayPal’s agentic commerce services emphasize fraud detection, buyer protection, dispute resolution, merchant integration, and compatibility across leading AI surfaces and payment protocols [12].

Market risk. Accenture reports that 78% of financial payments leaders expect fraud to increase significantly with agentic payments, 87% believe trust will be the key adoption barrier, and 60% lack a dedicated response plan with forensic tooling for agent-driven fraud [13]. The Consumer Bankers Association similarly highlights unresolved consumer protection and liability questions when autonomous agents initiate payments [14]. The implication is direct: if an agent pays without clear authorization, policy scope, counterparty validity, spend limit, or execution proof, someone owns the loss.

III. x402 Authorization Boundary

x402 exposes the authorization boundary cleanly. A client requests a resource, receives a 402 response with requirements, creates payment, resubmits with proof, and receives the resource after settlement [1, 3]. That makes it natural for agent tools, and it gives TrustLoopGuard a precise place to sit: after the paid resource asks for money, before the wallet signs.

Table 1: Market primitives and the corresponding TrustLoopGuard control.

External primitive	What it solves	TrustLoopGuard mapping
AP2 mandates	Prove user intent and transaction scope	Mandate scope, external mandate metadata, decision receipt.
x402 payment requirements	Communicate amount, asset, network, payee, and resource	Normalized payment action and payment-requirement hash.
AgentCore payment session	Bound and observe agent spend under concurrency	Payment session, atomic reservation, commit/rollback.
Visa/Mastercard agent identity	Let merchants and issuers recognize trusted agent activity	Principal binding, counterparty metadata, proof fields.
PayPal buyer protection/disputes	Handle post-payment failure and recovery	Outcome trail, dispute metadata, reversal capability.

The missing part is not settlement. Rails and facilitators can move money. The missing part is the pre-payment policy decision. A raw x402 client can ask a wallet to sign. A production agent-payment system needs to answer:

- Which principal is spending?
- Which mandate authorizes the spend?
- Is the rail, asset, network, endpoint, and `payTo` allowed?
- Is the amount below per-payment and per-session limits?
- Has the same requirement already been paid?
- Can concurrent payments overspend the session?
- Does the settlement proof match the authorized request?

TrustLoopGuard should sit in the gap between payment requirement and wallet signature. Authorization says the agent may hold a payment credential. TrustLoopGuard says whether this specific payment action should happen.

IV. System Model

The proposed system has five roles.

1. **Agent.** The AI workflow that requests a paid resource.

2. **Paid resource.** An API, MCP server, data feed, or content endpoint that returns Payment Required.
3. **TrustLoopGuard x402 interceptor.** A deterministic adapter around the agent's HTTP client.
4. **TrustLoopGuard authorization service.** The policy and ledger system that authorizes and reserves spend.
5. **Wallet/facilitator/provider.** The external system that signs, verifies, and settles payment.

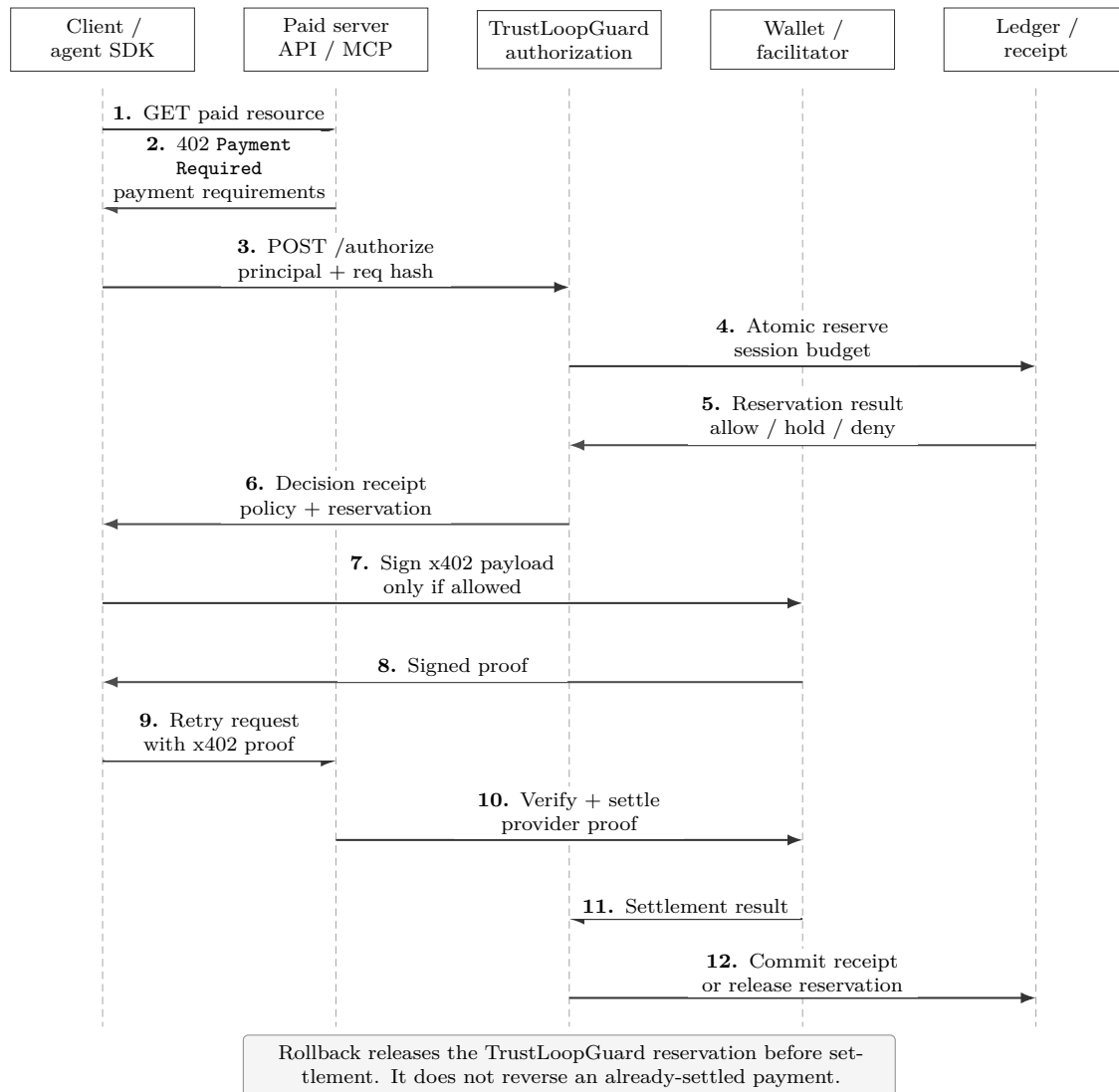


Figure 1: The x402 lifecycle as a sequence flow. TrustLoopGuard sits between the 402 payment requirement and wallet signing, reserves budget before payment, and commits only when settlement proof matches the authorized action.

The agent does not receive a general-purpose payment tool. It receives an HTTP client or tool wrapper. The wrapper handles x402 requirements deterministically and calls TrustLoopGuard before signing.

V. Authorization Primitives

V.1. Principal

The principal is the actor allowed to spend. It may be a user, service principal, agent identity, or workload identity:

```
spid:research-agent-prod
user:operator
agent:refund-bot
workspace:acme/procurement-agent
```

The x402 payTo address is not the principal. It is the counterparty. A key rule follows:

If the authenticated API key or workload token is bound to principal P , then the request's `principal_id` must equal P unless an explicit delegation is present and valid.

Without this check, an agent can claim another identity in the request body.

V.2. Mandate

A mandate is bounded delegated authority. It is the internal version of the idea that AP2 makes explicit: an agent should act under signed constraints, not ambient permission [4]. The design is also consistent with capability systems such as macaroons, where authority is attenuated by contextual caveats [15].

```
{
  "principal_id": "spid:research-agent",
  "scope": {
    "rails": ["x402"],
    "operations": ["paid_research_api"],
    "allowed_hosts": ["api.vendor.com"],
    "allowed_networks": ["eip155:8453"],
    "allowed_assets": ["USDC"],
    "max_per_payment_minor": 25000,
    "max_session_minor": 1000000,
    "expires_at": "2026-07-08T20:00:00Z"
  }
}
```

For AP2 compatibility, a later version can store open and closed mandate references, checkout hashes, payment hashes, signer key references, and external verification results.

V.3. Policy

Policy is the deterministic rule layer. It decides whether a normalized payment action is allowed, denied, or held for approval.

```
family: financial
when:
  agents: ["spid:research-agent"]
  action_kinds: ["payment"]
  rails: ["x402"]
controls:
  mandate_required: true
  max_amount_minor: 25000
  hold_above_minor: 100000
  allowed_counterparty_ids:
    - "x402:eip155:8453:0xmerchant"
```

The language above is intentionally policy-like rather than model-like. LLMs can propose a payment. They should not authorize payment.

V.4. Counterparty

The counterparty is the destination and business identity of the payment:

```
{
  "id": "x402:eip155:8453:0xmerchant",
  "kind": "api_provider",
  "display_name": "Market Data API",
  "metadata": {
    "host": "api.vendor.com",
    "resource": "GET /premium/quote",
    "network": "eip155:8453",
    "asset": "USDC",
    "payTo": "0xmerchant"
  }
}
```

This is where prompt injection often attacks: destination, amount, resource, asset, or network. TrustLoopGuard should bind the authorization to a normalized payment-requirement hash.

V.5. Reservation

Spend limits must be enforced atomically. A check-only design fails under concurrency:

```
read remaining budget
if enough, sign payment
write new balance later
```

The safer lifecycle is:

```
reserve budget atomically
sign payment
commit on success
rollback on failure
```

Reservation protects the budget while signing and settlement are in progress.

V.6. Execution Proof

Execution proof is the evidence that the external rail did what was authorized:

```
{
  "provider": "coinbase_x402_facilitator",
  "network": "eip155:8453",
  "asset": "USDC",
  "payTo": "0xmerchant",
  "amount": "0.25",
  "paymentRequirementHash": "sha256:...",
  "settlementReference": "0txhash",
  "paymentResponseHeader": "..."
}
```

TrustLoopGuard should not commit a payment as executed unless the proof matches the reserved action.

Table 2: Threats and required controls for x402 agentic payments.

Threat	Failure mode	Required control
Prompt injection changes destination	Agent pays wrong <code>payTo</code> or host	Counterparty allowlist, normalized requirement hash.
Parallel calls overspend session	Stale balance reads under concurrency	Atomic reservation and conditional budget update.
Agent claims wrong principal	Request body impersonates another agent	Auth-context principal binding.
Duplicate requirement is replayed	Same paid resource charged twice	Idempotency key and payment-requirement hash.
Proof does not match authorization	Settlement differs from reserved action	Commit-time proof verifier.
Provider failure after reservation	Budget remains locked incorrectly	Reservation expiry and rollback.

VI. x402 Authorization Lifecycle

The lifecycle below shows how TrustLoopGuard fits into the x402 request-response model without changing the rail.

Step 1: the agent receives a payment requirement. The research agent calls a paid API:

```
GET https://api.vendor.com/premium/quote
```

The server returns:

```
402 Payment Required
PAYMENT-REQUIRED: { amount, asset, network, payTo, resource, facilitator, ... }
```

Step 2: the interceptor normalizes the requirement. The SDK converts the x402 requirement into a canonical authorization request:

```
{
  "principal_id": "spid:research-agent",
  "operation": "paid_research_api",
  "rail": "x402",
  "amount": {
    "amount_minor": 25000,
    "currency": "USDC"
  },
  "counterparty": {
    "id": "x402:eip155:8453:0xmerchant",
    "kind": "api_provider",
    "metadata": {
      "host": "api.vendor.com",
      "resource": "GET /premium/quote",
      "network": "eip155:8453",
      "payTo": "0xmerchant"
    }
  },
  "mandate_id": "mandate_research_session",
  "payment_requirement_hash": "sha256:..."
}
```

Step 3: TrustLoopGuard authorizes and reserves. The authorization call is specific to agentic payments:

```
POST /v1/agentic-payments/authorize
```

An allowed response includes a reservation:

```
{
  "id": "apay_123",
  "decision": "authorized",
  "reservation_id": "res_123",
  "expires_at": "2026-07-08T17:15:00Z",
  "remaining_session_budget_minor": 975000,
  "decision_receipt_id": "dec_123"
}
```

A denied response remains auditable:

```
{
  "decision": "denied",
  "reason": "counterparty_not_allowed",
  "decision_receipt_id": "dec_124"
}
```

A held response creates an approval path:

```
{
  "decision": "held",
  "approval_request_id": "apr_123",
  "decision_receipt_id": "dec_125"
}
```

Step 4: the wallet signs only if authorized. The wallet or x402 client signs after receiving an authorized reservation. The agent does not directly control the signing key.

Step 5: TrustLoopGuard commits or rolls back. If settlement succeeds:

```
POST /v1/agentic-payments/apay_123/commit

{
  "settlement_reference": "0ctxhash",
  "provider": "coinbase_x402_facilitator",
  "payment_response": { "...": "..." }
}
```

If signing or settlement fails:

```
POST /v1/agentic-payments/apay_123/rollback

{
  "reason": "settlement_failed",
  "provider_error": "invalid_exact_evm_transaction_failed"
}
```

VII. Commit and Rollback Are Not Reversal

The term rollback can be dangerous if it is left ambiguous.

Thesis. Rollback releases a TrustLoopGuard reservation. It does not reverse an already-settled onchain payment.

Before payment, TrustLoopGuard can deny, hold, or reserve. During payment, TrustLoopGuard can prevent wallet signing if the action is not authorized. If signing or settlement fails, TrustLoopGuard can roll back its reservation. After settlement succeeds, TrustLoopGuard cannot magically undo the blockchain payment. Recovery becomes a separate action: provider refund, compensating payment, dispute, manual recovery, or loss recording.

This is why the control point must be pre-signing. Once a payment is signed and settled, the system moves from prevention to evidence, dispute, and recovery.

VIII. Developer Experience

The operator should not manually call three endpoints. They should install a deterministic x402 interceptor around the agent's HTTP client:

```
const guardedFetch = trustloop.agenticPayments.wrapX402(fetch, {
  principalId: "spid:research-agent",
  sessionId: "aps_research_2026_07_08"
});
```

The agent continues to call HTTP resources normally:

```
const result = await guardedFetch("https://api.vendor.com/premium/quote");
```

The SDK handles:

1. catching the 402 response,
2. normalizing the payment requirement,
3. calling TrustLoopGuard authorization,
4. reserving budget atomically,
5. invoking wallet signing only if allowed,
6. retrying with payment proof,
7. committing or rolling back,
8. storing receipt and outcome.

The dashboard should not be magic. It should show the payment request, principal, mandate, policy decision, counterparty, amount, session budget, settlement proof, trace, and outcome. If TrustLoopGuard recommends a cap, it should also show the inputs behind that cap: model or tool cost, resource, counterparty, session, and prior spend. Hidden metrics are hard to trust.

IX. Agent-Readable Integration Contract

TrustLoopGuard should not rely on an agent interpreting a whitepaper correctly. The authorization layer should publish a machine-readable contract that coding agents, SDKs, gateways, wallets, and orchestration systems can consume deterministically. The prose explains the architecture; the contract tells software what must be sent, what may be returned, and which lifecycle transitions are valid.

```
{
  "operation": "authorize_agentic_payment",
  "required_fields": [
    "principal_id",
    "mandate_id",
    "rail",
    "amount",
    "counterparty",
    "payment_requirement_hash"
  ],
  "decisions": ["authorized", "denied", "held"],
  "post_authorization_actions": ["commit", "rollback"],
  "proof_required_for_commit": true
}
```

This contract should exist as OpenAPI, JSON Schema, SDK types, and, where agents use tools directly, MCP-style tool schemas. A coding agent should not infer endpoint names, rollback semantics, or required proof fields from natural language. It should read the contract, generate a typed client, and preserve the lifecycle invariants.

1. Normalize the x402 payment requirement before policy evaluation.
2. Bind `principal_id` from the authenticated context or a signed delegation.
3. Reserve budget before wallet signing.
4. Sign only the payment requirement that matches the authorized hash.
5. Commit only when settlement proof matches the reserved action.
6. Roll back only an internal reservation; never describe rollback as payment reversal.

The same contract should be readable by humans and machines. A human can inspect why a payment was allowed, denied, held, committed, or rolled back. An agent can perform the integration without hallucinating a generic payment API or skipping the pre-signing control point.

X. Threat Model

Prompt injection changes destination. An untrusted page tells the agent to use a different paid endpoint or wallet destination. TrustLoopGuard should normalize `payTo`, host, resource, and network; enforce allowed counterparties; hash payment requirements; and deny proof mismatch.

Agent overspends through parallel calls. The agent launches many paid API calls at once against one budget. TrustLoopGuard should enforce atomic reservation, reservation expiry, conditional budget updates, commit, and rollback.

Agent claims wrong principal. The request body says `principal_id=user:operator`, but the API key belongs to another service. TrustLoopGuard should bind principal identity from the authentication context and reject mismatches unless a signed delegation exists.

Duplicate payment requirement. The same `x402` requirement is retried or replayed. TrustLoopGuard should derive idempotency from the normalized requirement hash, principal, and session.

Settlement proof does not match authorization. The proof settles a different amount, asset, network, or payee. TrustLoopGuard should commit only if the proof matches the reserved action, and record a failed outcome otherwise.

XI. Implementation Roadmap

Phase 1: authorization surface. Define the minimal agentic-payment authorization lifecycle:

```
POST /v1/agentic-payments/authorize
POST /v1/agentic-payments/{id}/commit
POST /v1/agentic-payments/{id}/rollback
GET  /v1/agentic-payments/{id}/receipt
```

The implementation should preserve the same domain model throughout the lifecycle: typed payment actions, mandates, policies, ledger reservations, receipts, and outcomes.

Phase 2: x402 rail. Add a first-class `x402` rail, payment requirement normalizer, normalized requirement hash, proof metadata schema, and an SDK/interceptor around common HTTP clients.

Phase 3: principal and mandate enforcement. Enforce API-key or workload principal binding. Add mandate fields for allowed hosts, `payTo` values, networks, assets, per-payment limits, session limits, and expiry.

Phase 4: atomic session budgets. Add payment sessions and reservations. Use transaction-safe conditional updates so concurrent payments cannot overspend the same

session. Reservations should expire and roll back if the provider never returns a successful proof.

Phase 5: proof and observability. Verify settlement proof against the reserved action. Store decision receipts, execution receipts, outcomes, and trace attributes. Use standard trace-context language for cross-system correlation [16].

Phase 6: AP2 compatibility. Store open and closed mandate references, checkout hashes, payment hashes, signer key references, and external verification results. Export evidence packages that can support dispute workflows.

XII. Limitations

This paper does not claim that TrustLoopGuard should process payments, custody funds, run a wallet, settle onchain transactions, or replace merchant checkout. It proposes an authorization layer. The external rail remains responsible for signing, verification, and settlement. PSPs and networks remain responsible for their own tokenization, fraud, dispute, and settlement rules.

The first implementation should also avoid overclaiming AP2 compatibility. AP2 support requires careful handling of signed mandates, checkout hashes, payment hashes, key binding, and receipts. The x402 integration can start with normalized payment requirements and internal mandates, then add AP2-compatible evidence export later.

XIII. Conclusion

Agentic payments do not mainly need a new way to move money. They need a way to prove that this specific money should move.

The market is fragmenting at the rail and commerce-protocol layers, but converging at the authorization layer. AP2 has mandates. x402 has payment requirements and proofs. AWS AgentCore has session budgets, reservation, and observability. Visa, Mastercard, and PayPal are adding identity, tokenization, controls, and dispute primitives. Accenture's market data shows that trust and fraud response are already board-level concerns.

TrustLoopGuard should own the missing independent layer:

authorization for agents that pay: cap the spend before it fires, reserve budget,
and prove what happened after.

That is the narrow layer. It is also the dataset that becomes valuable later: every allowed payment, blocked payment, escalation, rollback, proof mismatch, and loss event. Before anyone can underwrite agent risk, someone has to sit at the action layer and record what actually happened.

References

- [1] Coinbase Developer Platform. x402: The Payment Protocol for Agentic Commerce. <https://www.x402.org/x402-whitepaper.pdf>, 2025. Accessed 2026-07-08.
- [2] Coinbase Developer Platform. x402 Overview. <https://docs.cdp.coinbase.com/x402/welcome>, 2026. Accessed 2026-07-08.
- [3] Coinbase Developer Platform. How x402 Works. <https://docs.cdp.coinbase.com/x402/core-concepts/how-it-works>, 2026. Accessed 2026-07-08.
- [4] Google Agentic Commerce. Agentic Payment Protocol (AP2) Specification. <https://ap2-protocol.org/ap2/specification/>, 2026. Accessed 2026-07-08.
- [5] Google Agentic Commerce. Agent Payments Protocol Flow Examples. <https://ap2-protocol.org/ap2/flows/>, 2026. Accessed 2026-07-08.
- [6] Amazon Web Services. Technical Deep Dive: AgentCore Payments and Innovation in Agentic Commerce. <https://aws.amazon.com/blogs/machine-learning/technical-deep-dive-agentcore-payments-and-innovation-in-agentic-commerce/>, 2026. Accessed 2026-07-08.
- [7] Amazon Web Services. AgentCore CLI Payments Documentation. <https://github.com/aws/agentcore-cli/blob/main/docs/payments.md>, 2026. Accessed 2026-07-08.
- [8] Visa. Visa Intelligent Commerce. <https://www.visa.com/en-us/solutions/intelligent-commerce>, 2026. Accessed 2026-07-08.
- [9] Visa Developer Center. Trusted Agent Protocol. <https://developer.visa.com/capabilities/trusted-agent-protocol>, 2026. Accessed 2026-07-08.
- [10] Mastercard. Mastercard Unveils Agent Pay, Pioneering Agentic Payments Technology. <https://www.mastercard.com/us/en/news-and-trends/press/2025/april/mastercard-unveils-agent-pay-pioneering-agentic-payments-technology-to-power-commerce.html>, 2025. Accessed 2026-07-08.
- [11] Mastercard. Mastercard Launches Agent Pay for Machines to Unlock Super-Fast, Always-On Payments. <https://www.mastercard.com/us/en/news-and-trends/press/2026/june/mastercard-launches-agent-pay-for-machines.html>, 2026. Accessed 2026-07-08.
- [12] PayPal. PayPal Launches Agentic Commerce Services to Power AI-Driven Shopping. <https://newsroom.paypal-corp.com/2025-10-28-PayPal-Launches-Agentic-Commerce-Services-to-Power-AI-Driven-Shopping>, 2025. Accessed 2026-07-08.
- [13] Accenture. Agentic Commerce and the Future of Payments. <https://www.accenture.com/us-en/blogs/banking/agentic-commerce-payments>, 2026. Accessed 2026-07-08.
- [14] Consumer Bankers Association. Agentic AI Payments: Navigating Consumer Protection, Innovation, and Regulatory Frameworks. <https://consumerbankers.com/wp-content/uploads/2026/01/CBA-Agentic-Symposium-White-Paper-2026-01v2.pdf>, 2026. Accessed 2026-07-08.
- [15] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with Contextual Caveats for Decentralized

Authorization in the Cloud. In *Network and Distributed System Security Symposium*, 2014.

- [16] World Wide Web Consortium. Trace Context. <https://www.w3.org/TR/trace-context/>, 2021. W3C Recommendation.